

## 2.1 Time Evolution Equation:

$$x_t = \Psi \cdot x_{t-1} + \epsilon_p$$

$x_t$ : It is the state of the object at the current time step. It contains,  $x = [x, y, v_x, v_y, a_x, a_y]^T$

$\Psi$ : It is the state transition Matrix which defines the physical model of motion.

$x_{t-1}$ : It is the state vector from the previous time step.

$\epsilon_p$ : This is process noise which accounts for the uncertainty.

## 2.2 Measurement Equation:

$$z_t = \phi x_t + \epsilon_m$$

$z_t$ : This is the measurement vector, consists of observed data at time step  $t$ .

$\phi$ : Measurement Matrix, which maps the state space to the measurement space.

$x_t$ : It is the state vector, represents true state of the object.

$\epsilon_m$ : It relates to the noise inherent in the sensors.

### 3.2 Analysis of Lag Size effects:

Increasing the lag size  $L$ , improves the accuracy and smoothness of the state estimates because the filter consists a larger window of future measurements to correct the estimate. By observing the trajectory into the future, the smoother can ~~at~~ better distinguish between transient sensor noise and actual system dynamics.

As  $L$  grows, the correlation between the current state and distant future measurements weakens due to accumulated process noise. Also, the computational complexity increases and the system introduces a longer delay before providing final estimates.

#### 4.1 . Non-linear Components:

i]  $g[x_t]$  : Component 1:  $x_{t+1} = x_t + \Delta t \cdot v_t \cos[\theta_t]$   
 Component 2:  $y_{t+1} = y_t + \Delta t \cdot v_t \sin[\theta_t]$

Reason: They contain trigonometric functions  $\cos[\theta_t]$ ,  $\sin[\theta_t]$ , making the relationship non-linear.

ii]  $h[x_t]$  : There are no non-linear components.

#### 4.2 . Jacobian of $g[x_t]$ :

$$\therefore g[x_t] = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \end{bmatrix} = \begin{bmatrix} x_t + \Delta t \cdot v_t \cdot \cos[\theta_t] \\ y_t + \Delta t \cdot v_t \cdot \sin[\theta_t] \\ \theta_t \\ v_t \end{bmatrix}$$

Jacobian of  $g[x_t] = J_g$

$$\therefore J_g = \frac{\partial g}{\partial x} = \begin{bmatrix} \frac{\partial g_1}{\partial x} & \frac{\partial g_2}{\partial y} & \frac{\partial g_1}{\partial \theta} & \frac{\partial g_1}{\partial v} \\ \frac{\partial g_2}{\partial x} & \frac{\partial g_2}{\partial y} & \frac{\partial g_2}{\partial \theta} & \frac{\partial g_2}{\partial v} \\ \frac{\partial g_3}{\partial x} & \frac{\partial g_3}{\partial y} & \frac{\partial g_3}{\partial \theta} & \frac{\partial g_3}{\partial v} \\ \frac{\partial g_4}{\partial x} & \frac{\partial g_4}{\partial y} & \frac{\partial g_4}{\partial \theta} & \frac{\partial g_4}{\partial v} \end{bmatrix}$$

$$\therefore J_g = \begin{bmatrix} 1 & 0 & -\Delta t \cdot v_t \cdot \sin[\theta_t] & \Delta t \cos[\theta_t] \\ 0 & 1 & -\Delta t \cdot v_t \cdot \cos[\theta_t] & \Delta t \sin[\theta_t] \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.3 . Jacobian of  $h[x_t] = J_h$

$$\therefore h[x_t] = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \end{bmatrix}$$

$$\therefore J_h = \frac{\partial h}{\partial x} = \begin{bmatrix} \frac{\partial h_1}{\partial x} & \frac{\partial h_1}{\partial y} & \frac{\partial h_1}{\partial \theta} & \frac{\partial h_1}{\partial v} \\ \frac{\partial h_2}{\partial x} & \frac{\partial h_2}{\partial y} & \frac{\partial h_2}{\partial \theta} & \frac{\partial h_2}{\partial v} \end{bmatrix}$$

$$\therefore J_h = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

4.6 : It is better to use the extended Kalman filter in this case because the system model is non linear.

The unicycle motion model defines the state update for position as :

$$x_{t+1} = x_t + \Delta t \cdot v_t \cdot \cos[\theta_t]$$

$$y_{t+1} = y_t + \Delta t \cdot v_t \cdot \sin[\theta_t]$$

Here, the trigonometric function creates non-linear relationship between the current state and the next state.

The standard Kalman filter assumes that the system is linear, it cannot directly process the trigonometric relationship.

Therefore, solution with EKF works by linearizing the non-linear function around the current state estimate using Jacobian Matrix.

4.8: Increasing  $\delta_t$  to  $\delta_t = [0.05, 0.05]$ :

When we increase the measurement noise to higher noise, the estimated trajectory becomes smoother and deviates more from individual data points compared to the low noise case.

This is because of how the Kalman filter trust between the model and the measurements:

- i] Measurement Noise Matrix  $[R]$ :  $\delta_t$  populates the  $R$  matrix, by increasing  $\delta_t$ , the filter assumes that sensor ~~are~~ readings are unreliable.
- ii] Kalman Gain: when  $R$  is high, Kalman gain decreases i.e., less weight on the measurement, high weight on the prediction.